

$$\mathbf{R}_s = \begin{bmatrix} 1 & \rho_{12} & \cdots & \rho_{1p} \\ \rho_{21} & 1 & \cdots & \rho_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ \rho_{p1} & \rho_{p2} & \cdots & 1 \end{bmatrix}, \quad (2.53)$$

onde ρ_{ij} é o coeficiente de Spearman específico entre as variáveis X_i e X_j .

2.5.3.4 Visualização Das Responsabilidades Estimadas Pelo GMM

Plotar as responsabilidades estimadas por um modelo probabilístico é fundamental para abrirmos entendemos como o algoritmo se comporta ao longo do espaço amostral.

Matematicamente, a responsabilidade de uma componente k sobre uma observação $\mathbf{x}_i$ é dada por:

$$\gamma_{ik} = P(z_i = k \mid \mathbf{x}_i), \quad (2.54)$$

onde z_i é a componente latente da observação i , e γ_{ik} quantifica o nível de pertencimento dessa amostra $\mathbf{x}_i$ à respectiva componente k .

Se uma dessas componentes estiver diretamente ligada à nossa classe de interesse (como a classe positiva), podemos usar essas responsabilidades para desenhar um mapa probabilístico de pertencimento.

Esses gráficos deixam claro como o modelo espalha suas incertezas e certezas pelo espaço. Em cenários unidimensionais, conseguimos ilustrar esse comportamento com uma curva de responsabilidade. Na segunda dimensão, podemos gerar um mapa de calor (ou de contorno), colorindo o plano conforme a intensidade da probabilidade calculada. Já em dimensões mais altas, o caminho mais comum é utilizar gráficos em 3D ou a combinação das variáveis no 2d.

A grande vantagem dessas representações está na interpretação das fronteiras de decisão. Elas mostram nitidamente onde o classificador assume maior ou menor segurança, ajudando a validar a estrutura probabilística capturada pelo modelo, além de escancarar zonas de ambiguidade e o comportamento de dados atípicos.

2.5.4 Seleção De Variáveis

2.5.4.1 Busca Exaustiva Combinatória Para Seleção De Variáveis

A busca exaustiva combinatória é uma estratégia de seleção baseada em uma premissa simples, mas poderosa: testar absolutamente todas as combinações de variáveis dentro de um limite de tamanho predefinido para comparar seus desempenhos em vez de

avaliar os atributos isoladamente ou um a um, o método examina o cenário completo. Isso garante uma segurança muito maior para identificar quais subconjuntos de fato otimizam os resultados do modelo.

Diferente dos métodos sequenciais tradicionais nos quais os atributos entram ou saem do modelo de forma gradual, a busca exaustiva faz um pente fino direto em todas as combinações permitidas. Essa abordagem blindar a análise contra o risco de deixar um arranjo altamente promissor passar despercebido.

2.5.4.2 GSFS

A GSFS, *Generalized Sequential Forward Selection*, por sua vez, introduz uma abordagem sequencial e inteligente para mitigar essa barreira. A ideia central é expandir o subconjunto de variáveis de maneira gradual, mas sem a restrição de incluir apenas uma variável por vez. Em cada iteração, o método analisa blocos candidatos compostos por r variáveis que ainda estão disponíveis. Cada bloco é acoplado temporariamente ao conjunto já selecionado nas etapas anteriores e o desempenho dessa nova combinação é medido por uma função de avaliação. O grupo que injeta o maior ganho de eficiência é promovido definitivamente para o subconjunto selecionado.

Partindo do mesmo conjunto de variáveis originais:

$$\mathcal{X} = \{X_1, X_2, \dots, X_p\}. \quad (2.55)$$

O algoritmo inicia os trabalhos com um subconjunto de seleção completamente vazio:

$$\mathcal{S}_0 = \emptyset. \quad (2.56)$$

Ao alcançar a etapa t , o modelo toma como base o subconjunto acumulado até ali, denotado por $\mathcal{S}_t$. Com os atributos que restaram no banco de dados, estruturam-se os grupos candidatos $\mathcal{G}_r$, cada um contendo exatamente r variáveis:

$$\mathcal{G}_r \subseteq \mathcal{X} \setminus \mathcal{S}_t, \quad |\mathcal{G}_r| = r. \quad (2.57)$$

Para testar o potencial de cada grupo candidato, simula-se um novo subconjunto composto, unindo o que já havia sido selecionado ao novo bloco em teste:

$$\mathcal{C} = \mathcal{S}_t \cup \mathcal{G}_r. \quad (2.58)$$

Em seguida, o algoritmo calcula o *score* correspondente a esse arranjo combinado:

$$Score(\mathcal{S}_t \cup \mathcal{G}_r). \quad (2.59)$$

O bloco escolhido para integrar o modelo na etapa t é aquele que maximiza a função critério adotada:

$$\mathcal{G}_r^* = \arg \max_{\mathcal{G}_r \subseteq \mathcal{X} \setminus \mathcal{S}_t} Score(\mathcal{S}_t \cup \mathcal{G}_r). \quad (2.60)$$

Definido o melhor grupo, o subconjunto de variáveis selecionadas é oficialmente atualizado para a rodada seguinte:

$$\mathcal{S}_{t+1} = \mathcal{S}_t \cup \mathcal{G}_r^*. \quad (2.61)$$

Esse ciclo se repete de forma contínua até esbarrar em algum critério de parada preestabelecido, que pode ser o alcance do número máximo de atributos desejado, a estagnação do **score** (ausência de melhora significativa) ou o esgotamento do limite computacional configurado.

Dessa forma, a GSFS atua como um excelente meio-termo entre uma busca exaustiva global e uma seleção puramente individual. Ao avaliar grupos de variáveis de maneira síncrona em vez de recursos isolados, o método ganha velocidade sem abrir mão de capturar a sinergia entre os atributos.

Esse aspecto é vital, pois certas variáveis podem parecer pouco expressivas quando analisadas sozinhas, mas ganham enorme poder preditivo quando combinadas. Em cenários complexos como a detecção de fraudes, por exemplo, padrões suspeitos raramente se isolam em um único comportamento; eles costumam emergir justamente da interação oculta entre diferentes dados. A GSFS oferece essa resposta equilibrada, mesmo que segundo Pudil, Novovičová e Kittler (1993) isso ocorra à custa de um tempo de computação muito mais longo, especialmente com o aumento dos valores dos parâmetros, além de não haver uma maneira sem ser testando de determinar os valores corretos dos parâmetros a fim de obter o melhor subconjunto .

3 Metodologia

3.1 Caracterização da pesquisa

3.1.1 Natureza da pesquisa

Este trabalho se define como uma pesquisa aplicada. A escolha por essa vertente se justifica pelo próprio objetivo do estudo: explicar e usar o ferramental científico para intervir em um problema real e complexo: a segurança e a detecção de fraudes no sistema financeiro. Trata-se, portanto, de colocar a ciência de dados e a estatística a serviço de uma necessidade prática, sem abrir mão do rigor metodológico e das demonstrações matemáticas que validam cada passo da análise.

3.1.2 Abordagem metodológica

Para dar conta desse desafio, o caminho escolhido foi a abordagem quantitativa. Isso significa que a matéria-prima são os dados numéricos e as métricas estatísticas, processados por modelos computacionais. É importante salientar o quanto a lente matemática é fundamental, pois com ela, conseguimos medir e comparar matematicamente o desempenho de cada cenário, separando impressões subjetivas de resultados concretos de classificação.

Somado a isso, o trabalho ganha um contorno essencialmente experimental. A pesquisa não se limita a rodar um algoritmo padrão; ela desenha um ambiente de testes onde testa e confronta diferentes arranjos; Varia os hiperparâmetros dos modelos, muda as combinações de variáveis e experimenta técnicas distintas de projeção matemática para entender quais dessas estruturas conseguem capturar com maior precisão o comportamento sutil das fraudes em cartões de crédito.

3.2 Ferramentas e ambiente computacional

3.2.1 Importância

Detalhar o *hardware* e as versões dos pacotes utilizados é o que garante a reprodutibilidade do estudo, permitindo que outros pesquisadores rodem o mesmo código e entendam a relação entre o tempo de processamento e os recursos disponíveis.

3.2.2 Ambiente de execução

Todas as etapas de modelagem e processamento foram executadas localmente. Para isso, foi utilizado um *notebook* Acer Nitro AN515-55 equipado com o sistema operacional Windows 10. A máquina conta com uma arquitetura de 64 bits, processador Intel Core i5 da décima geração e cerca de 8 GB de memória RAM ativa, distribuídos em uma estrutura de 4 núcleos físicos e 8 núcleos lógicos de processamento.

3.2.3 Linguagem de programação e ambiente interativo

A espinha dorsal de todo o código do projeto foi escrita em Python (versão 3.13.3). A escolha pela linguagem foi natural, dado o seu protagonismo indiscutível no ecossistema de ciência de dados e aprendizado de máquina; Para que esses testes acontecessem de forma organizada e visual, os scripts foram desenvolvidos dentro do ambiente interativo do Jupyter Notebook, contando com o suporte dos motores *IPython* e *IPykernel*. Essa dinâmica de desenvolvimento em blocos foi crucial para que pudéssemos investigar os resultados parciais de cada algoritmo e documentar o raciocínio analítico passo a passo.

Tabela 1 – Ambiente Python utilizado

Item	Versão
Python	3.13.3
Notebook	7.4.2
IPython	9.2.0
IPykernel	6.29.5

Fonte: Elaborado pelo autor.

3.2.4 Bibliotecas utilizadas

Para colocar os dados em ordem e realizar as operações matemáticas mais densas, conta-se com a eficiência das bibliotecas *pandas* e *NumPy*.

A aplicação dos modelos e o cálculo das métricas de validação são feitas pela biblioteca *scikit-learn*, uma ferramenta de referência para o aprendizado de máquina. Para a criação de gráficos compreensíveis, recorreremos ao trio *matplotlib*, *seaborn* e *plotly*. Por fim, pacotes especializados como *SciPy* (para suporte estatístico), *tqdm* (para acompanhar o progresso dos loops mais demorados) e *openTSNE* (para a redução dimensional não linear) completaram a caixa de ferramentas computacionais.

Tabela 2 – Principais bibliotecas utilizadas

Biblioteca	Versão
<i>pandas</i>	2.2.3
<i>NumPy</i>	2.2.5
<i>scikit-learn</i>	1.7.2
<i>matplotlib</i>	3.10.5
<i>seaborn</i>	0.13.2
<i>plotly</i>	6.7.0
<i>SciPy</i>	1.15.3
<i>ipykernel</i>	6.29.5
<i>tqdm</i>	4.67.3
<i>openTSNE</i>	1.0.4

Fonte: Elaborado pelo autor.

3.2.5 Repositório do projeto

Para que este trabalho não seja apenas um documento estático, mas um ponto de partida para outros estudos, toda a estrutura de códigos, tabelas auxiliares e relatórios interativos foi organizada em um repositório digital público. Essa transparência garante que qualquer pessoa possa acessar, auditar e reproduzir fielmente os caminhos tomados nesta pesquisa.

O código-fonte completo e os artefatos do projeto estão disponíveis em:

<<https://github.com/Vitor-Craveiro-Unesp/Bachelor-s-Thesis-in-Statistics---2026>>

3.2.6 Ferramentas de escrita e documentação

A escrita e a formatação deste documento foram totalmente desenvolvidas em L^AT_EX. Essa escolha técnica garantiu que toda a estrutura de fórmulas matemáticas, tabelas analíticas e citações bibliográficas seguisse um padrão visual rigoroso, limpo e profissional. Além disso, manter a documentação teórica alinhada com os testes interativos dos cadernos de código evitou qualquer distorção entre a teoria descrita e os resultados práticos obtidos.

3.3 Base de dados

3.3.1 Origem da base de dados

Os dados reais que dão vida a este trabalho vêm da plataforma Kaggle, extraídos do célebre repositório *Credit Card Fraud Detection* (Machine Learning Group - ULB, 2018).